Tranig Studio

הסבר על הקבצים train_deeplab.py ו-run_deeplab.py

מה הם? שני סקריפטים שהתוכנה יוצרת בלחיצה על `Create Environment & Folders`.
`train_deeplab.py` מאמן מודל, ו-`run_deeplab.py` מריץ מודל מאומן על תמונה. אפשר לפתוח אותם בעורך של Vision Studio, לשנות את ההגדרות בראש הקובץ, לשמור (`Save Code`) ולהריץ (`Run ...`).

החלק המשותף לשני הקבצים

חלק	מה הוא עושה
`import ...`	טוען את הספריות (PyTorch, NumPy, Pillow). אם הן חסרות – מודפסת הודעה ללחוץ על Install Requirements.
`MEAN / STD`	ערכי נרמול הצבעים שהמודל המאומן מראש מצפה להם. לא לשנות.
`build_model()`	בונה את DeepLabV3 (ResNet-50 או MobileNetV3) ומחליף את השכבה האחרונה כך שתתאים למספר הסוגים שלך.
`to_input()`	מקטין את התמונה ל-`IMG_SIZE` והופך אותה למספרים שהמודל מקבל.
`load_trained()`	טוען מודל מאומן מקובץ `.pth`, כולל שמות הסוגים והגודל שבו אומן.
`predict()`	מריץ את המודל על תמונה ומחזיר לכל פיקסל את מספר הסוג. פיקסל שהביטחון בו נמוך מ-`CONF` נחשב רקע.

train_deeplab.py – אימון

ההגדרות בראש הקובץ – כאן משנים

```
DATASET_DIR = 'my_dataset'          # folder with images/ masks/ classes.txt
ARCH = 'DeepLabV3 ResNet-50'        # or 'DeepLabV3 MobileNetV3'
EPOCHS = 50                         # passes over all the data
BATCH = 2                           # images per step (min 2)
LR = 0.005                          # learning rate
OUT_PATH = 'model.pth'              # where the trained model is saved
CLASSES = []                        # empty = read classes.txt
IMG_SIZE = 512                      # training image size
```

הגדרה	הסבר והמלצה
`DATASET_DIR`	תיקיית הנתונים. לאימון על חלוקה: `'my_dataset/split/train'` .
`ARCH`	ResNet-50 – מדויק יותר. MobileNetV3 – מהיר וחסכוני בזיכרון.
`EPOCHS`	יותר = לומד יותר, אבל לוקח זמן. התחל ב-30–50 ובדוק את התוצאה.
`BATCH`	ב-RTX 2080 עם ResNet-50 – להשאיר 2. עם MobileNetV3 אפשר 4–8.
`LR`	מומלץ להשאיר. אם ה-loss קופץ או הופך ל-`nan` – להקטין ל-`0.001` .
`CLASSES`	להשאיר ריק – הסוגים נקראים מ-`classes.txt` .
`IMG_SIZE`	אובייקטים קטנים (רחפן רחוק) – גודל גדול עוזר. אם נגמר הזיכרון – `384` .

מה קורה כשמריצים – שלב אחרי שלב

1. `load_classes()` – קורא את שמות הסוגים.
2. `list_samples()` – מוצא תמונות שיש להן מסכה ב-`masks/` . תמונות בלי מסכה מדולגות.
3. `SegmentationDataset` – לכל תמונה: טוען תמונה ומסכה ומקטין את שתיהן ל-`IMG_SIZE` .
4. `build_model(..., True)` – בונה מודל ממשקולות מאומנות מראש (בפעם הראשונה – הורדה של כ-160MB).
5. לולאת האימון: בכל Epoch עובר על כל התמונות, מחשב את השגיאה (`loss`) ומתקן את המודל.
6. בסוף – שומר את המודל ב-`OUT_PATH` .

מה רואים בקונסולה

```
Device: cuda                          # cuda = graphics card, cpu = processor
Found 120 images with masks
Classes: ['background', 'drone']
Epoch 1/50 - loss: 0.6302
Epoch 2/50 - loss: 0.3410             # loss should go down
...
Saved model to model.pth
```

איך לקרוא את ה-loss: המספר צריך לרדת לאורך האימון. אם הוא נתקע גבוה – צריך יותר תמונות מתויגות או יותר Epochs. אם הוא `nan` – להקטין את `LR` .

run_deeplab.py – הרצת מודל על תמונה

ההגדרות בראש הקובץ

```
MODEL_PATH = 'model.pth'              # trained model
IMAGE_PATH = 'test.jpg'               # image to test
OUT_PATH = 'test_pred.jpg'            # result image
CONF = 0.5                            # minimum confidence (0-1)
COLORS = [(224, 122, 95), ...]        # color per class
```

הגדרה	הסבר והמלצה
`MODEL_PATH`	המודל שהאימון יצר.
`IMAGE_PATH`	התמונה לבדיקה. אפשר נתיב מלא, למשל `'C:/Tranig/photos/a.jpg'` (עם `/`).
`OUT_PATH`	שם קובץ התוצאה.
`CONF`	גבוה (0.7) – פחות זיהויים שגויים, אבל עלול לפספס. נמוך (0.3) – תופס יותר, עם יותר רעש.
`COLORS`	הצבע הראשון לסוג הראשון ב-`classes.txt` , השני לשני וכן הלאה.

מה קורה כשמריצים

1. `load_trained()` – טוען את המודל, שמות הסוגים והגודל שבו אומן.
2. פותח את התמונה ומריץ `predict()` – מתקבלת מפה: לכל פיקסל מספר סוג.
3. מדפיס לכל סוג שנמצא איזה אחוז מהתמונה הוא תופס ובאיזה צבע.
4. `overlay()` – צובע את האזורים שזוהו בשכבה שקופה למחצה ושומר ל-`OUT_PATH` .

מה רואים בקונסולה

```
drone (#E07A5F): 2.35% of pixels
Saved: test_pred.jpg
```

אם לא נמצא כלום: `No objects found above confidence 0.50` – נסה להוריד את `CONF` , או לאמן עם יותר תמונות.

הפרמטרים לעומק

EPOCHS – כמה פעמים עוברים על הנתונים

Epoch אחד = המודל ראה את **כל** התמונות המתויגות פעם אחת. עם 100 תמונות ו-`BATCH = 2`, כל Epoch הוא 50 צעדים, ובכל צעד המודל מתקן את עצמו קצת. כמו לימוד למבחן – כל חזרה מחדדת.

```
Epoch 1/50 - loss: 0.63
Epoch 10/50 - loss: 0.21
Epoch 30/50 - loss: 0.09
Epoch 50/50 - loss: 0.08
```

`loss` = מדד השגיאה; נמוך יותר = טוב יותר.

- **ה-loss עדיין יורד בסוף** – המודל לא סיים ללמוד, להעלות.
- **ה-loss נעצר** – הגעת למקסימום, עוד Epochs רק מבזבזים זמן.
- **Overfitting** – יותר מדי Epochs על מעט תמונות: המודל "משנן" את התמונות שראה ונחלש על תמונות חדשות.

המלצה: להתחיל ב-50 ולבדוק על תמונות שהמודל **לא ראה**. מעט תמונות? עדיף פחות Epochs ויותר תיוג.

CONF – רף הביטחון

המודל נותן לכל פיקסל ציון ביטחון לכל סוג. דוגמה עם `CONF = 0.5`:

- פיקסל א': 92% צמחייה ← **צמחייה** (מעל הרף).
- פיקסל ב': 45% רכב ← **רקע** (מתחת לרף – המודל לא מספיק בטוח).

ערך	תוצאה	מתי
גבוה (0.7–0.9)	רק מה שבטוח – פחות טעויות, עלול לפספס	כשחשוב שלא יהיו התראות שווא
בינוני (0.5)	איזון	נקודת התחלה
נמוך (0.2–0.3)	תופס יותר, עם יותר "רעש"	כשחשוב לא לפספס (למשל מכשולים)

- **CONF לא משנה את המודל** – רק את פרשנות התוצאה. אפשר לשחק איתו בלי לאמן מחדש.
- קיים ב-`run_deeplab.py` (`CONF = 0.5`) וב-Auto-Labeling (`Min Conf %`, באחוזים). בתיוג אוטומטי – עדיף ערך קצת גבוה.

טיפ: להריץ את אותה תמונה עם 0.3, 0.5 ו-0.7 ולהשוות.

IMG_SIZE – גודל התמונה שהמודל רואה

כל תמונה מוקטנת לריבוע של `IMG_SIZE × IMG_SIZE` לפני שהמודל רואה אותה; בהרצה התוצאה מוגדלת חזרה לגודל המקורי. מקטינים כי תמונה מלאה צורכת המון זיכרון ומאטה, וכל התמונות ב-Batch צריכות להיות באותו גודל.

ערך	יתרון	חיסרון
384	מהיר, חסכוני בזיכרון	פרטים קטנים נעלמים
512	איזון טוב	ברירת המחדל
768–640	עצמים קטנים וגבולות חדים	איטי, יותר זיכרון

דוגמה: רכב רחוק של 20 פיקסלים בתמונה של 1920 – אחרי הקטנה ל-512 נשארים כ-5 פיקסלים; ב-768 – כ-8.

- אזורים גדולים (צמחייה, דרך, שמיים) – 512 מספיק, אפילו 384.
- עצמים קטנים או רחוקים – לנסות 640.
- RTX 2080 עם ResNet-50 – עד 640 עם `BATCH = 2`; מעבר לזה כנראה `CUDA out of memory`.
- הגודל נשמר בתוך `model.pth` – ההרצה משתמשת בו אוטומטית. שינוי = אימון מחדש.
- תמונה רחבה "נמחצת" לריבוע – לא מפריע, כי אותו עיוות קורה באימון ובהרצה.

ARCH – ה"מוח" שרואה את התמונה

ל-DeepLabV3 שני חלקים: **Backbone** (שלד) שמזהה צורות, קווים ומרקמים – זה מה ש-ARCH קובע; ו**ראש DeepLab** שמחליט לכל פיקסל לאיזה סוג הוא שייך – זהה בשתי האפשרויות.

	ResNet-50	MobileNetV3
גודל	גדול (~40 מיליון פרמטרים)	קטן (~11 מיליון)
דיוק	גבוה יותר	טוב, קצת פחות
מהירות אימון	איטי יותר	מהיר פי כמה
זיכרון בכרטיס	הרבה (BATCH 2)	מעט (BATCH 4–8)
מהירות הרצה	בינונית	מהיר, מתאים לזמן אמת
גבולות עדינים / עצמים קטנים	טוב יותר	פחות

שניהם **מאומנים מראש** על מיליוני תמונות – הם כבר "יודעים לראות", והאימון שלך רק מלמד את הסוגים שלך. לכן מספיקות עשרות עד מאות תמונות.

- **ResNet-50** – כשחשוב דיוק ויש זמן לאימון (ברירת המחדל).
- **MobileNetV3** – ניסויים מהירים, כשנגמר הזיכרון, או להרצה על וידאו בזמן אמת.
- המודל זוכר באיזו ARCH אומן – משנים רק לפני אימון.

טיפ: לאמן קודם עם MobileNetV3 כדי לוודא מהר שהתיוג בכיוון הנכון, ואת הגרסה הסופית עם ResNet-50.

טיפים:

- שינויים בעורך נשמרים רק אחרי `Save Code` .
- הקבצים לא נדרסים ב-Create Environment אם הם כבר קיימים – אפשר לשנות אותם בבטחה.
- כדי להתחיל מחדש מהגרסה המקורית – מחק את הקובץ ולחץ שוב Create Environment & Folders.